Week 5: Building the Frontend: React 1

Problem Statement

This week you will begin building the frontend for **ThreadHive**, a social media platform where users can create communities (subreddits), post discussion topics (threads), add comments, vote and interact with content.

In this assignment, you will create an **authentication UI** that includes **Login functionality**, navigation through a **Header component**, and conditional rendering in the **main App component**.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
3. As you make changes throughout this tutorial, remember to regularly commit your work.

These are the main components of the project:

```
src
├── App.jsx          Root component of the Application
├── main.jsx         An entry point that renders the App component
├── pages/           Page components (Login, Register, etc.)
└── components/      Reusable UI components (Header, Footer, etc.)

tests/             Unit tests for React components
```

This session is structured in two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Login Page	No AI Tools
Part 2	AI-Assisted Development	Custom Instructions for React, Building Auth Forms, Multimodal Prompts, Unit Tests	GitHub Copilot

In **Part 1** you will build a React component manually, without AI assistance. This ensures you understand the core React concepts.

In **Part 2** we will use GitHub Copilot for different tasks of frontend development: generating code for new components, setting up custom instructions for React code generation, using multimodal prompts with screenshots to guide UI code generation, and generating unit tests for React components.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

By the end of this session, you will be able to:

- Build **controlled forms** in React and manage user input with `useState`
- Use **props** and **conditional rendering** to create dynamic UIs
- Set up **custom instructions** to guide Copilot for frontend development
- Use **multimodal prompts** with screenshots to guide UI code generation
- Generate **unit tests** for React components using AI assistance

Part 1: Manual Implementation of React Components

In this part, you will build a simple authentication UI with a **Login page**, and conditional rendering done with `useState`. Working through this by hand will give you a solid understanding of React's core building blocks before you use AI to generate similar code in Part 2.

1.1 Project Setup

Open a terminal in the root of the repository and run the following commands:

```
npm install
```

This should install all the necessary dependencies. Start the development server to confirm everything is working:

```
npm run dev
```

Open `http://localhost:5173` in your browser. Right now you should see a blank page since we haven't implemented any components yet.

1.2 Building the Login Component

Implement the **Login Component** where users can enter their email and password. The component is a **controlled form** — React state is the single source of truth for the form's input values.

Requirements:

1. Define `email` and `password` state variables for form fields using `useState`.
2. Create a login form inside a container element with email and password input fields, and a submit button.
3. On form submission, log the entered email and password to the console, and show a browser alert confirming successful login.

1.3 Wiring in the Login page in App.jsx

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

`App.jsx` is the root component. It owns the `currentPage` state and decides which page to render using **conditional rendering**.

Conditional Rendering

Conditional rendering in React works just like conditional statements in JavaScript. The most common patterns are:

```
// Using a ternary operator
{
  isLoggedIn ? <Dashboard /> : <Login />;
}

// Using && (renders nothing if condition is false)
{
  showBanner && <Banner />;
}

// Using if/else before the return
function App() {
  if (currentPage === "login") return <Login />;
  if (currentPage === "register") return <Register />;
  return <div>Unknown page</div>;
}
```

Implementing App.jsx

1. Use a state variable `currentPage` (via `useState`) to track the active page. Set its initial value to `'login'`.
2. Conditionally render components based on the `currentPage` value:
    - `"login"` → render `<Login />`
    - `"register"` → render `<Register />` (already implemented component)

Open `http://localhost:5173`. You should see the login page rendered. Change the initial value of `currentPage` to `'register'`. Refreshing the page should show the register page.

Commit your work with a relevant message.

Part 2: Building with GitHub Copilot

In Part 1, you built a React authentication UI manually. In this part, you will use GitHub Copilot to:

1. Set up custom instructions so Copilot always generates React code that matches your conventions
2. Build Header & Footer components, and a navigation system using **Agent Mode**.
3. Build a Reset password page using **Agent Mode**

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. Style components using **multimodal image prompts** (screenshots as design references)
5. Generate unit tests using a custom Agent.

2.1 Custom Instructions for the Frontend

In previous weeks you used `AGENTS.md` (or the equivalent `.github/copilot-instructions.md`) to guide Copilot for the backend. Now that we are building a React frontend, we will build instructions with frontend-specific conventions.

Use the `/init` command to generate a new `AGENTS.md` instructions file for this frontend project. In the Agent mode in Copilot Chat panel, select a powerful reasoning model (like Opus 4.7) type:

```
/init
```

Carefully review the generated instructions file. It should include guidelines for React development, such as using functional components (not Class components), state and styling conventions. We will update and customize these instructions in the following weeks as the project evolves.

Commit your instructions file to the repository.

2.3 Building Header & Footer and Navigation with Agents

Let us build the Header & Footer components and set up navigation using GitHub Copilot in Agent Mode. We will prompt Copilot to generate these components and integrate them into our app's layout.

Open a new chat in **Agent Mode** and use the following prompt to generate the Header & Footer components along with the navigation system.

```
Update the existing `src/components/Header/Header.jsx` and
`src/components/Footer/Footer.jsx` components (and their corresponding CSS
files).

1. The Header should include buttons for ""Login", "Register", and
"Logout".
2. The Header component should receive a prop `onNavigate` from `App.jsx`
to handle navigation.
3. `isAuthenticated` is just a dummy variable that will be eventually
replaced with real authentication logic. For now, keep it set to `false` to
show Login and Register buttons. If `isAuthenticated` is `true`, the Logout
button should be displayed.
4. The button clicks should do the following (temporary behavior until
authentication is implemented):
   - `Login` → calls `onNavigate('login')`
   - `Register` → calls `onNavigate('register')`
   - `Logout` → calls `onNavigate('login')`
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
5. The Footer should include basic copyright information with the current
year and the app name (ThreadHive).

Integrate these components into the main `App.jsx` layout so that the
Header appears at the top and the Footer at the bottom of every page.
```

Review the generated components to ensure they meet the requirements and integrate correctly into the `App.jsx` layout. Test the navigation buttons to verify that they trigger the appropriate `onNavigate` calls and that the Header and Footer are consistently displayed across all pages.

2.3 Implementing React Components with GenAI

Now let us now add a **Reset Password** page to the app. Since this is a new task, let us start a NEW chat in **Agent Mode**.

1. Enter the following prompt:

   ```
   Create a Reset Password component at `pages/Auth/ResetPassword.jsx`.

   The component should:

   - Be a controlled form with fields for email, old password, new
   password, and confirm password
   - Accept an onResetPassword callback prop that App can use to respond
   to password reset
   - On submit, validate that newPassword and confirmPassword match and
   call onResetPassword callback if validation passes. Show an error
   message below the form if validation fails. Show a success message
   using conditional rendering after successful password reset
   ```

   Review the generated code to ensure it satisfies the requirements.

2. Update your `App.jsx` to include this new page in the navigation and conditional rendering logic. You can add a new button in the `Header` for "Reset Password" that navigates to this page.

3. Test the new page in the browser. Try submitting the form with mismatched passwords to see the error message, and then with matching passwords to see the success message.

4. Commit your changes with a relevant message.

2.4 Multimodal Inputs: Prompting with Images

So far we have been prompting Copilot with text only. GitHub Copilot also supports AI models that can handle **multimodal inputs** — you can attach images directly to your prompts. This is especially powerful

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by davidj@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

for UI work, where a screenshot communicates layout and visual hierarchy far more precisely than a text description.

Note: Not all models support multimodal inputs, though an increasing number of AI models do. To check which models support images, select 'Manage Language Models' in the model dropdown in the Chat panel. (or type in the command Ctrl+Shift+P / Cmd+Shift+P → 'Manage Language Models'). The models tagged with 'Vision' capability can process images.

Prompting from a Reference Site Screenshot

In this exercise, we will use a screenshot of a login page and use it as a design reference to improve our `Login` component.

Exercise: Style Login Using a Reference Screenshot

1. In Copilot Chat panel switch to **Agent Mode**.
2. Start a **new chat session** (`Ctrl+N` / `Cmd+N`).
3. Attach the screenshot to your prompt:

- Drag and drop the image in /resources/login-screenshot.png into the chat input, **or**
- Click the **paperclip icon** in the chat input and select the login-screenshot.png file

4. Add the following prompt alongside the image:

```
Using the design of the login page in this screenshot, update our
Login component to use a similar layout.

For the image on the right, download and use any placeholder image
from https://unsplash.com/ and place it in the `resources` folder.
Adapt the layout to our existing CSS Modules approach.
Do not clone the site's exact colors or typography — just use the
layout structure as inspiration.
```

5. Review the generated changes. Notice how Copilot interprets the spatial layout from the screenshot — you did not have to describe "flexbox column on the left with buttons stacked vertically" in words.

Iterating with a Follow-up Image Prompt

Multimodal prompting is most powerful as an iterative conversation. You can attach new images in follow-up messages:

1. Take a second screenshot focusing on a specific detail — for example, the login form.

2. In the **same chat session** (do not start a new one — Copilot retains the context of your previous edits), attach the new screenshot and type:

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
Move the labels inside the input fields as placeholders. Retain the
font styling of the label.
```

3. Observe how Copilot builds on the previous conversation context when making this new targeted change.

Key Takeaway: A screenshot communicates spatial relationships and visual hierarchy in a way that text cannot. Use image prompts whenever you want Copilot to work from a visual reference rather than relying entirely on verbal descriptions.

2.5 Generating Unit Tests for React Components

Let us now use GitHub Copilot to generate unit tests for the new components we have built. We will create a custom Agent that generates tests for our React Components. We will use **Vitest** as our test runner and **React Testing Library** for testing React components.

We can create a custom agent using the `/create-agent` command. Start a new chat session and enter the following prompt:

```
/create-agent Create an agent called 'react-testing-agent' that creates
unit tests for React components. Use 'React Testing Library' as the testing
library and Vitest as the test runner. Keep all the tests in a separate
top-level tests/ directory. Keep the agent workspace scoped.
```

This will create a new agent that we can use to generate tests for our React components. Review the agent instructions to ensure it is set up correctly. Save the agent (it should be saved in `.github/agents/react-testing-agent.agent.md`).

We are now ready to generate tests using this agent.

Exercise : Generate Unit Tests for Login component

1. Open a new chat session. Select the `react-testing-agent` in the Agent dropdown.
2. Add `pages/Auth/Login.jsx` as context.
3. Enter the following prompt:

```
Create Unit tests for the Login component. The tests should cover the
following scenarios:

1.  A render test – confirm the email and password fields are all
present in the page when the component is rendered.
2.  An input capture test – type into each field and confirm the
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
values are reflected in the inputs (controlled component behavior)
3.  A successful submission test – fill in the email and password
fields and confirm that the form submits the correct data.
```

Review the generated tests. Make sure they are testing the correct scenarios and that they are well-structured.

4. Run the tests to confirm they pass:

   ```
   npm run test
   ```

5. If a test fails, paste the error message into the Copilot Chat in the same session:

   ```
   The following test is failing: [paste the error here].
   Look at the Register component and fix the test to match its actual
   implementation.
   ```

6. [Optional] You can similarly generate unit tests for the other components like the Registration and Reset Password components.

Commit your tests to the repository with a relevant message.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

This file is meant for personal use by david@davidystephenson.com only.
Sharing or publishing the contents in part or full is liable for legal action.